Unit 4 Progress Check: FRQ Part B

1. The GameAccount class is used to store information about players of an online game. A partial declaration of the GameAccount class is shown.

```
public class GameAccount
{
  /**
   *  Returns the username of the player
   */
  public String getUsername()
  { /* implementation not shown */ }

  /**
   *  Returns the age of the player
   */
  public int getAge()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The GamePlatform class maintains an ArrayList of GameAccount objects named allPlayers. A partial declaration of the GamePlatform class is shown.

```
public class GamePlatform
{
  /** An ArrayList of all player accounts */
  public ArrayList<GameAccount> allPlayers;

  /**
   *  Returns the average age of players in allPlayers whose age
   *  is between 13 and 19, inclusive
   *  Preconditions: allPlayers contains at least one player
   *                 whose age is between 13 and 19.
   *                 allPlayers is not null and contains no null
   *                 elements.
   *  Postcondition: allPlayers is unchanged.
   */
  public double avgTeenAccounts()
  { /* to be implemented */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

Unit 4 Progress Check: FRQ Part B

Write the GamePlatform method avgTeenAccounts. The method should return the average age of the players in allPlayers whose age is between 13 and 19, inclusive.

For example, suppose allPlayers has the following contents:

Username	"star_guy"	"a_knight"	"dino_dan"	"cool_cat"	"happy_hero"	"fox_sox"
Age	23	14	12	15	13	19

For these contents of allPlayers, the method avgTeenAccounts should return 15.25. There are four players whose age is between 13 and 19, inclusive ("a_knight", "cool_cat", "happy_hero", and "fox_sox"). The average age of these four players is $(14 + 15 + 13 + 19)/4$, which is 15.25.

Complete method avgTeenAccounts.

```
/**
 * Returns the average age of players in allPlayers whose age
 * is between 13 and 19, inclusive
 * Preconditions: allPlayers contains at least one player
 *                whose age is between 13 and 19.
 *                allPlayers is not null and contains no null
 *                elements.
 * Postcondition: allPlayers is unchanged.
 */
public double avgTeenAccounts()
```

Point 1

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all elements in `allPlayers` (no bounds errors)

Canonical Solution:

```
public double avgTeenAccounts()
{
   int sumAges = 0;
   int countTeens = 0;

   for (GameAccount account : allPlayers)
   {
      int age = account.getAge();
```

Unit 4 Progress Check: FRQ Part B

```
      if (age >= 13 && age <= 19)
      {
         sumAges += age;
         countTeens++;
      }
   }
   return (double) sumAges / countTeens;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to use the accessed value
- return early, as long as bounds and indices otherwise support accessing all elements

Responses will not earn the point if they

- use an indexed loop with correct bounds but fail to access elements inside the loop
- access the elements of `allPlayers` incorrectly
- use array syntax to access `ArrayList` elements

Point 2

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Calls `getAge` on an element from the list

Canonical Solution:

```
public double avgTeenAccounts()
{
   int sumAges = 0;
   int countTeens = 0;

   for (GameAccount account : allPlayers)
   {
      int age = account.getAge();
      if (age >= 13 && age <= 19)
```

Unit 4 Progress Check: FRQ Part B

```
      {
         sumAges += age;
         countTeens++;
      }
   }
   return (double) sumAges / countTeens;
}
```

0	✓ 1

The response earns the point.

Responses can still earn the point even if they

- use array syntax to access `ArrayList` elements

Responses will not earn the point if they

- ever call a `GameAccount` method incorrectly

Point 3

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Identifies players aged 13 to 19

Canonical Solution:

```
public double avgTeenAccounts()
{
   int sumAges = 0;
   int countTeens = 0;

   for (GameAccount account : allPlayers)
   {
      int age = account.getAge();
      if (age >= 13 && age <= 19)
      {
         sumAges += age;
         countTeens++;
      }
```

Unit 4 Progress Check: FRQ Part B

```
   }
   return (double) sumAges / countTeens;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- call `GameAccount` or `ArrayList` methods incorrectly

Responses will not earn the point if they

- fail to call `getAge` on an element from the list

Point 4

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Counts identified players

Canonical Solution:

```
public double avgTeenAccounts()
{
   int sumAges = 0;
   int countTeens = 0;

   for (GameAccount account : allPlayers)
   {
      int age = account.getAge();
      if (age >= 13 && age <= 19)
      {
         sumAges += age;
         countTeens++;
      }
   }
   return (double) sumAges / countTeens;
}
```

Unit 4 Progress Check: FRQ Part B

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- incorrectly identify players

Point 5

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Returns average of identified players (algorithm)

Canonical Solution:

```
public double avgTeenAccounts()
{
   int sumAges = 0;
   int countTeens = 0;

   for (GameAccount account : allPlayers)
   {
      int age = account.getAge();
      if (age >= 13 && age <= 19)
      {
         sumAges += age;
         countTeens++;
      }
   }
   return (double) sumAges / countTeens;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

Unit 4 Progress Check: FRQ Part B

- call `GameAccount` or `ArrayList` methods incorrectly
- use array syntax to access `ArrayList` elements
- fail to call `getAge` on an element from the list
- incorrectly identify players

Responses will not earn the point if they

- fail to return the computed value
- fail to calculate the average as a `double`
- modify the list

Unit 4 Progress Check: FRQ Part B

2. The Game class is used to store the scores of a home team and an away team in a game. A partial declaration of the Game class is shown.

```
public class Game
{
  /**
   *  Returns the home team's score for this game
   */
  public int getHomeScore()
  { /* implementation not shown */ }

  /**
   *  Returns the away team's score for this game
   */
  public int getAwayScore()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The Season class maintains an ArrayList of Game objects named allGames. A partial declaration of the Season class is shown.

```
public class Season
{
  /** The list of all games played during the season */
  private ArrayList<Game> allGames;

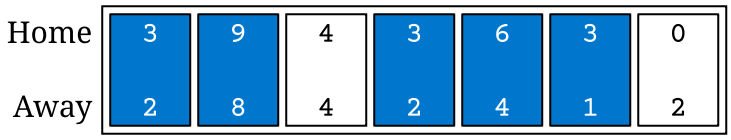
  /**
   *  Returns the number of games in the longest streak of
   *  consecutive games where the home team scored more than
   *  the away team
   *  Preconditions: allGames is not empty.
   *                 No elements of allGames are null.
   *  Postcondition: allGames is unchanged.
   */
  public int getLongestHomeWinStreak()
  { /* to be implemented */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

Unit 4 Progress Check: FRQ Part B

Write the Season method getLongestHomeWinStreak. The method should return the number of games that make up the longest winning streak by the home team. A winning streak is a number of consecutive games won by a team. A game is considered a win for the home team if the home team score is greater than the away team score.

For example, suppose allGames contains the following Game objects. The shaded elements represent home team wins.



For these contents of allGames, the method getLongestHomeWinStreak() should return 3 because the longest winning streak for the home team is 3 games.

Complete method getLongestHomeWinStreak.

```
/**
 * Returns the number of games in the longest streak of
 * consecutive games where the home team scored more than
 * the away team
 * Preconditions: allGames is not empty.
 *                No elements of allGames are null.
 * Postcondition: allGames is unchanged.
 */
public int getLongestHomeWinStreak()
```

Point 1

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all elements in `allGames` (no bounds errors)

Canonical Solution:

```
public int getLongestHomeWinStreak()
{
   int maxStreak = 0;
   int currentStreak = 0;

   for (Game g : allGames)
   {
      if (g.getHomeScore() > g.getAwayScore())
```

Unit 4 Progress Check: FRQ Part B

```
      {
         currentStreak++;
         if (currentStreak > maxStreak)
         {
            maxStreak = currentStreak;
         }
      }
      else
      {
         currentStreak = 0;
      }
   }
   return maxStreak;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to use the accessed value
- return early, as long as bounds and indices otherwise support accessing all elements

Responses will not earn the point if they

- use an indexed loop with correct bounds but fail to access elements inside the loop
- access the elements of the list incorrectly
- use array syntax to access `ArrayList` elements

Point 2

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Calls `getHomeScore` and `getAwayScore` on an element from the list

Canonical Solution:

```
public int getLongestHomeWinStreak()
{
   int maxStreak = 0;
```

Unit 4 Progress Check: FRQ Part B

```
   int currentStreak = 0;

   for (Game g : allGames)
   {
      if (g.getHomeScore() > g.getAwayScore())
      {
         currentStreak++;
         if (currentStreak > maxStreak)
         {
            maxStreak = currentStreak;
         }
      }
      else
      {
         currentStreak = 0;
      }
   }
   return maxStreak;
}
```

0	✓ 1

The response earns the point.

Responses can still earn the point even if they

- use array syntax to access `ArrayList` elements

Responses will not earn the point if they

- call either method incorrectly

Point 3

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Determines whether a home score is greater than an away score

Canonical Solution:

Unit 4 Progress Check: FRQ Part B

```
public int getLongestHomeWinStreak()
{
   int maxStreak = 0;
   int currentStreak = 0;

   for (Game g : allGames)
   {
      if (g.getHomeScore() > g.getAwayScore())
      {
         currentStreak++;
         if (currentStreak > maxStreak)
         {
            maxStreak = currentStreak;
         }
      }
      else
      {
         currentStreak = 0;
      }
   }
   return maxStreak;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- call `Game` or `ArrayList` methods incorrectly

Responses will not earn the point if they

- fail to call `getHomeScore` or `getAwayScore`

Point 4

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Initializes count and increments it within a conditional statement within a loop

Unit 4 Progress Check: FRQ Part B

Canonical Solution:

```
public int getLongestHomeWinStreak()
{
   int maxStreak = 0;
   int currentStreak = 0;

   for (Game g : allGames)
   {
      if (g.getHomeScore() > g.getAwayScore())
      {
         currentStreak++;
         if (currentStreak > maxStreak)
         {
            maxStreak = currentStreak;
         }
      }
      else
      {
         currentStreak = 0;
      }
   }
   return maxStreak;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to identify all home wins
- return early

Responses will not earn the point if they

- fail to initialize the count correctly or reset it within a loop

Point 5

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

Unit 4 Progress Check: FRQ Part B

- Returns maximum number of consecutive home wins (algorithm)

Canonical Solution:

```
public int getLongestHomeWinStreak()
{
   int maxStreak = 0;
   int currentStreak = 0;

   for (Game g : allGames)
   {
      if (g.getHomeScore() > g.getAwayScore())
      {
         currentStreak++;
         if (currentStreak > maxStreak)
         {
            maxStreak = currentStreak;
         }
      }
      else
      {
         currentStreak = 0;
      }
   }
   return maxStreak;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- call `Game` or `ArrayList` methods incorrectly
- stop loop early or late due to incorrect bounds
- use array syntax to access `ArrayList` elements

Responses will not earn the point if they

- fail to return the computed value
- return an incorrect value due to early return
- modify the list

Unit 4 Progress Check: FRQ Part B

- use incorrect logic to determine when to increment the count